M09 — Prácticas: despliegue integral de AceriaApp en Docker Desktop para Windows

ARQUITECTURA HEXAGONAL Y MICROSERVICIOS · PRÁCTICAS MÓDULO 9 · JAIME GALLO

En los módulos anteriores se ha construido AceriaApp de forma progresiva: API REST, separación hexagonal, patrones de arquitectura, DDD, comunicación asíncrona, resiliencia y publicación mediante YARP. En esta práctica no vamos a programar otra versión del sistema ni a reemplazar su lógica de negocio. Vamos a desplegar el resultado de M08 en contenedores, observar sus dependencias y comprobar que continúa funcionando cuando dejamos de ejecutar las APIs directamente en Windows.

El equipo del alumno utiliza **Windows**, **PowerShell 7** y **Docker Desktop con WSL 2 y contenedores Linux**. Esta distinción es esencial: utilizar imágenes Linux no convierte el curso en un curso de administración de Linux. Se conservarán el código y los proyectos de M01–M08. El material de M09 añade únicamente Dockerfile, Compose, configuración de laboratorio y un script de verificación.

El recorrido tiene tres momentos. Primero se distingue dónde se usó Docker en los módulos anteriores. Después se preparan SQL Server y RabbitMQ cuando las aplicaciones siguen funcionando en Windows. Finalmente se construyen y ejecutan **seis contenedores**, se comprueba su colaboración a través del gateway y se ensayan la interrupción y la recuperación de servicios. El criterio de éxito no es que aparezca la palabra «running»: hay que obtener respuestas HTTP correctas, observar una proyección generada por mensajería y recuperar datos persistidos tras un reinicio.

Modo de trabajo: ejecuta los fragmentos desde PowerShell 7 en el directorio que se indica. Conserva la misma sesión en los pasos 9.8–9.15, porque utiliza variables definidas anteriormente (`$gateway`, `$ordenId` y `$eventoId`). Los proyectos acabados de `PROYECTO/` sirven para comprobar resultados, no para sustituir la realización de los ejercicios previos.

1. Qué hace Docker en cada módulo y por qué

Docker no aparece por primera vez en el despliegue integral. En los módulos previos sirvió, según el caso, para suministrar una dependencia de pruebas o para levantar el broker de mensajería, mientras los procesos .NET permanecían en Windows. Conviene identificar esa diferencia antes de continuar, porque determina qué dirección de conexión y qué herramienta de diagnóstico corresponden a cada proceso.

Módulos	Uso de Docker	Dónde se ejecutan las APIs	Qué verifica el alumno
M01	No se necesita	Windows, con LocalDB para persistencia	API, datos y pruebas propias del módulo
M02–M05	Opcional: SQL Server Express de pruebas a través de Testcontainers	Windows	Integración con SQL Server real y resultados xUnit
M06	RabbitMQ mediante Compose; SQL Server puede participar en pruebas mediante Testcontainers	Windows	Publicación outbox, recepción inbox y proyección persistente
M07	Broker heredado de M06	Windows	Salud, degradación y recuperación
M08	Broker y dependencias de M06–M07		Enrutamiento, balanceo y entrega de eventos

Módulos	Uso de Docker	Dónde se ejecutan las APIs	Qué verifica el alumno
		Windows, incluidas las instancias de YARP e Inventario	
M09	SQL Server, RabbitMQ y las cuatro instancias .NET dentro de Compose	Contenedores Linux administrados desde Windows	Funcionamiento integral, resistencia a fallos y persistencia

La tabla presenta **tres formas de trabajo** que no son intercambiables. En la primera, Windows ejecuta la aplicación y LocalDB se utiliza desde el propio Windows. En la segunda, las APIs siguen siendo procesos `dotnet run` de Windows pero se conectan al RabbitMQ que Docker publica en `localhost:5672`. En la tercera, los procesos .NET y las dependencias viven en una red privada de Compose: allí, `localhost` señala al contenedor que realiza la petición y los otros componentes se localizan por su nombre de servicio.

Los laboratorios A y B permiten reconocer la configuración anterior sin cambiar de arquitectura. El laboratorio C despliega la etapa final `M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga` y proporciona la evidencia central de M09. No se utiliza SQLite ni un proveedor `InMemory` para simplificar artificialmente la prueba de persistencia.

2. Preparación verificable del equipo Windows

Comprueba primero que el ordenador puede ejecutar contenedores Linux desde Docker Desktop. Necesitas Windows 10 u 11 compatible con Docker Desktop, virtualización habilitada, WSL 2, PowerShell 7 y el SDK de .NET 8. Visual Studio con la carga de trabajo de ASP.NET Core permite inspeccionar los proyectos, aunque el despliegue se realizará desde la terminal. Para SQL Server y los demás contenedores reserva memoria y almacenamiento suficientes; como referencia inicial, deja **15–20 GB libres** y varios gigabytes de memoria disponibles para Docker, ajustando estos valores a los recursos reales del equipo.

Abre Docker Desktop, selecciona el motor de contenedores Linux basado en WSL 2 y, en una terminal **PowerShell 7 de Windows**, ejecuta:

```
wsl --status
wsl --update
docker version
docker compose version
docker info --format '{{.OSType}}'
docker run --rm hello-world
dotnet --version
```

Las órdenes contestan preguntas diferentes. `wsl --status` permite comprobar el subsistema de virtualización; `docker version` confirma que el cliente puede hablar con el motor; `docker compose version` acredita la herramienta que interpretará los servicios; y `docker info --format '{{.OSType}}'` debe mostrar `linux`. La imagen `hello-world` sirve como ensayo mínimo del motor, pero todavía no demuestra que se puedan iniciar SQL Server, RabbitMQ o AceriaApp. `dotnet --version` confirma la disponibilidad del SDK en el equipo anfitrión.

Si el cliente Docker existe pero no encuentra el servidor, abre Docker Desktop y espera a que termine el arranque. Si WSL 2 no está preparado, completa su instalación o actualización y reinicia Windows cuando sea necesario. No continúes con los laboratorios mientras `docker version` falle: en ese estado, cualquier error posterior se mezclaría con un problema previo del entorno.

Con el motor operativo, distingue cuatro conceptos. La **imagen** contiene una aplicación y sus dependencias empaquetadas; el **contenedor** es una ejecución concreta de esa imagen; el **servicio Compose** declara cómo iniciar esa ejecución y conectarla con las demás; el **volumen** almacena datos que deben sobrevivir a la desaparición de un contenedor. Los veremos materializados, y no solo definidos, en el laboratorio C.

3. Laboratorio A — SQL Server real para los tests de M02–M05

Los ejercicios de M02–M05 se apoyan en SQL Server para comprobar operaciones que un proveedor de base de datos en memoria no reproduce fielmente. Cuando se ejecutan tests de integración con **Testcontainers 3.10.0**, el propio conjunto de pruebas puede crear un SQL Server Express temporal, esperar a que esté listo y destruir sus recursos al finalizar. La imagen de referencia del curso es `mcr.microsoft.com/mssql/server:2022-CU14-ubuntu-22.04`.

En este laboratorio **no metemos la API en Docker**: el comando `dotnet test` sigue ejecutándose en Windows. Sitúate en una etapa completa, por ejemplo `M02_APIS_REST_ASPNET_CORE/PROYECTO/2.4-documentacion-openapi`, dentro de la carpeta que contiene su solución `AceriaApp.sln`. Quita primero cualquier conexión de pruebas heredada de otra terminal o laboratorio:

```
Remove-Item Env:\ACERIA_TEST_SQLSERVER -ErrorAction SilentlyContinue
docker info --format '{{.OSType}}'
dotnet test .\AceriaApp.sln -c Release
docker ps -a
```

La primera instrucción hace que las pruebas puedan utilizar el mecanismo de Testcontainers, siempre que así esté configurado en esa etapa. Mientras se ejecuta `dotnet test`, abre otra terminal para observar la creación del contenedor. Al terminar, lee el resumen real de xUnit: **el número de tests ejecutados, fallidos y omitidos** importa más que la mera ausencia de una excepción de compilación. `docker ps -a` permite inspeccionar el estado final, aunque la limpieza automática puede haber retirado ya los recursos efímeros.

Si no vas a utilizar Docker en esta fase, el curso contempla una alternativa propia de Windows: SQL Server Express LocalDB. Comprueba que se ha instalado y ejecuta desde la misma carpeta de solución:

```
sqllocaldb start MSSQLLocalDB
$env:ACERIA_TEST_SQLSERVER = 'Server=(localdb)\MSSQLLocalDB;Database=master;Integrated
Security=True;TrustServerCertificate=True'
dotnet test .\AceriaApp.sln -c Release
```

La variable `ACERIA_TEST_SQLSERVER` modifica el destino de conexión que utilizan los tests; no instala SQL Server ni crea por sí sola el esquema. LocalDB pertenece al anfitrión Windows y **no se encuentra dentro de los contenedores Linux**. Esta diferencia será decisiva cuando configuremos la API de Producción en el laboratorio C.

Repite la prueba deliberadamente con el motor Docker detenido, pero solo después de guardar el resultado correcto inicial. Con la cadena LocalDB configurada, los tests deberían poder utilizar la instancia local si la etapa admite ese camino y los requisitos están satisfechos; sin ella, una ejecución que necesite Testcontainers debe informar del problema y no dar un éxito ficticio. Deja registradas la alternativa utilizada y la salida de las pruebas. No cambies el proveedor EF Core a SQLite o `InMemory` para obtener un resultado favorable.

4. Laboratorio B — RabbitMQ con Docker, aplicaciones en Windows

En M06 aparece una segunda forma de utilizar contenedores: RabbitMQ funciona dentro de Docker, pero Producción e Inventario siguen arrancándose como procesos .NET en Windows. M07 hereda el mismo broker para practicar la recuperación ante fallos, y M08 lo combina con el gateway y las dos instancias de Inventario. El archivo `compose.yml` preparado en M06 publica los puertos `5672` (AMQP) y `15672` (interfaz web) sobre `127.0.0.1` y conserva los datos en un volumen.

Abre una terminal en la etapa con la que estés trabajando; para observar la solución final puedes utilizar `M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga`. Comprueba el archivo y levanta únicamente el broker:

```
Test-Path .\compose.yml
docker compose -f .\compose.yml config --quiet
docker compose -f .\compose.yml up -d rabbitmq
docker compose -f .\compose.yml ps
docker compose -f .\compose.yml exec -T rabbitmq rabbitmq-diagnostics -q ping
Test-NetConnection localhost -Port 5672
Test-NetConnection localhost -Port 15672
```

Hay aquí tres comprobaciones distintas: el archivo de Compose debe ser válido, el proceso RabbitMQ debe responder a su propio diagnóstico y los puertos deben ser accesibles desde Windows. Después abre `http://localhost:15672/` y entra con las credenciales **exclusivamente didácticas** `curso / curso-local`. Que se vea la consola de administración no basta para asegurar que una orden recorra todo el flujo outbox/inbox: eso se comprobará por HTTP y contra la proyección persistida.

Al ejecutar las APIs desde Windows, el cliente AMQP se conecta al puerto publicado en el anfitrión. Configura las variables en **cada PowerShell donde inicies una API**:

```
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:Comunicacion__RabbitMqUri = 'amqp://curso:curso-local@localhost:5672/'
$env:Comunicacion__WorkersActivos = 'true'
$env:Comunicacion__Prefijo = 'aceria-m08-manual'
```

Estas variables solo tienen efecto en los procesos iniciados desde la misma terminal. Completa las cadenas de conexión LocalDB que prescribe el punto concreto de M06–M08, y arranca Producción e Inventario en terminales separadas mediante `dotnet run --project ... --no-launch-profile`. En M08 se añade el proceso YARP y una segunda instancia de Inventario con los puertos descritos en el apartado 8.4. No confundas los nombres de carpeta y de proyecto con una dirección de red: el proceso Windows apunta a `localhost:5672` porque Docker está publicando precisamente ese puerto.

Una vez recibida una orden y observada su proyección, detén RabbitMQ con `docker compose stop rabbitmq`. La API puede seguir viva y, al mismo tiempo, comunicar que no está preparada para operar con mensajería. Recupera el broker con `docker compose start rabbitmq` y repite una operación nueva. Para interpretar un fallo utiliza `docker compose logs --tail 80 rabbitmq` y las comprobaciones de readiness de las APIs. Evita `down -v`: destruiría el volumen y haría imposible demostrar que el estado sobrevive a la interrupción.

Finalizado el laboratorio B, detén el Compose anterior antes de iniciar el laboratorio C. Ambos publican RabbitMQ en el mismo puerto de Windows y no pueden utilizar `127.0.0.1:5672` simultáneamente.

5. Laboratorio C — despliegue integral de M08 con Docker Compose

Aquí cambia el lugar de ejecución de las aplicaciones, **no su comportamiento de negocio**. Se toma la etapa final `M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga`: las mismas rutas REST, los mismos modelos de dominio, la mensajería real y el mismo gateway YARP. El proyecto M09 facilita los archivos de infraestructura para empaquetar esos ejecutables y conectarlos entre sí.

Compose define seis servicios. `sqlserver` aloja varias bases de datos SQL Server 2022 Express, cada una utilizada por el contexto correspondiente, y persiste su información en `sql-datos`. `rabbitmq` ofrece el broker AMQP, su consola web y el volumen `rabbit-datos`. `produccion` ejecuta la API de Producción y el publicador de eventos outbox. `inventario-1` e `inventario-2` ejecutan el mismo proyecto de Inventario contra la misma base de datos; esto permite comprobar el balanceo real de YARP. `gateway` es la entrada prevista para el cliente y reenvía las peticiones a los servicios apropiados.

El navegador del alumno usará principalmente `http://localhost:5100` para entrar por el gateway. El Compose publica también los puertos `5105`, `5106` y `5107` exclusivamente en el loopback de Windows para realizar diagnósticos directos; en un despliegue productivo no tendrían por qué exponerse. SQL Server escucha solo en el puerto **1433 privado de la red Docker**. Los componentes .NET se comunican entre sí por nombres como `produccion`, `inventario-1` y `rabbitmq`.

Presta atención a la persistencia: `(localdb)\MSSQLLocalDB` existe en Windows, no dentro de Linux. En M09 se utiliza `Server=sqlserver,1433` y autenticación SQL mediante las variables de entorno de Compose. EF Core sigue utilizando **SQL Server**, las clases C# no cambian y cada aplicación recibe la cadena que le corresponde. Del mismo modo, las rutas `localhost:5672` de procesos Windows pasan a ser `rabbitmq:5672` entre contenedores.

5.1 Preparar parámetros y construir las imágenes

Trabaja desde la **raíz del repositorio** y entra una sola vez en la carpeta del proyecto de despliegue. Copiarás el ejemplo de configuración a un archivo local privado antes de arrancar SQL Server:

```
Set-Location .\M09_DESPLIEGUE_DOCKER\PROYECTO
Copy-Item .\.env.example .\.env
notepad .\.env
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml config --quiet
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion preparar
```

En `.env` sustituye el ejemplo de `ACERIA_SA_PASSWORD` por una contraseña propia, compleja y adecuada a las reglas de SQL Server. No subas ese archivo a GitHub y no copies contraseñas en las capturas del ejercicio. La orden `docker compose ... config --quiet` valida la sintaxis del conjunto y la resolución de sus variables sin imprimir la configuración completa. La acción `preparar` confirma que el motor Docker y el archivo Compose están listos; aún **no ha arrancado los seis servicios**.

Para generar las imágenes, Compose utiliza `docker/Dockerfile.Servicio`. Su etapa de compilación parte de `mcr.microsoft.com/dotnet/sdk:8.0` y publica cada proyecto real de M08. La etapa final parte de `mcr.microsoft.com/dotnet/aspnet:8.0` y solamente contiene lo necesario para ejecutar la aplicación. El argumento `PROYECTO` selecciona Producción, Inventario o Gateway; el contexto de construcción apunta a la **raíz del repositorio**, de forma que se encuentran las referencias entre proyectos. En el Paso 9.2 compilarás uno de ellos antes de levantar el conjunto.

5.2 Iniciar paso a paso y ejecutar pruebas HTTP

Con `.env` guardado y Docker Desktop funcionando, inicia la secuencia automatizada desde `M09_DESPLIEGUE_DOCKER/PROYECTO`:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion levantar
```

El script primero inicia SQL Server y RabbitMQ y espera a que SQL responda. Después construye y levanta **Inventario 1, Inventario 2, Producción y Gateway**, en ese orden. La separación entre las dos instancias de Inventario evita que intenten preparar simultáneamente el mismo esquema de base de datos. Un servicio recién iniciado no se considera preparado hasta que se comprueba su endpoint de salud.

Tras el arranque, la automatización comprueba la vida del gateway, las dependencias de las APIs, el stock, el balanceo, la creación de una orden DDD de dos líneas, la planificación con respuesta HTTP `202`, el rechazo de una segunda planificación con `409` y la recepción asíncrona de su evento. Finalmente reinicia las aplicaciones y vuelve a consultar la misma orden y la misma proyección. Si cualquiera de estas verificaciones falla, la ejecución debe acabar con resultado `FAIL`, no dar por válido el despliegue porque los contenedores aparezcan como `Up`.

Puedes repetir la comprobación funcional sin rehacer las imágenes:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion verificar
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml ps
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml logs --tail 100 produccion inventario-1
inventario-2 gateway
```

La acción `verificar` genera una **orden de prueba nueva** y no sustituye ni borra las anteriores. La salida de `ps` muestra la situación de los contenedores; los logs permiten relacionar cada fallo con su proceso; y el reporte `.qa/docker-windows-e2e.json` conserva resultados e identificadores de la comprobación automática. El stock inicial `Acero = 300` se refiere a la base didáctica recién creada. Los datos de los volúmenes pueden persistir de una ejecución a otra: no presupongas que cada `verificar` devuelve el laboratorio a un estado vacío.

5.3 Persistencia, recuperación y limpieza controlada

Detener un contenedor y borrar sus datos son operaciones distintas. La configuración separa deliberadamente los procesos de los volúmenes SQL y RabbitMQ, de forma que podamos comprobar que una orden ya guardada se recupera tras reiniciar la aplicación.

Cuando quieras **detener** la ejecución sin eliminar contenedores ni volúmenes, utiliza:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion detener
```

Para retirar los contenedores y la red de Compose, conservando los volúmenes y permitiendo un nuevo despliegue con los mismos datos:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion bajar
```

La opción siguiente **destruye los volúmenes** del laboratorio. Es apropiada solamente para comenzar otra práctica con bases y colas vacías, una vez archivadas todas las evidencias:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion limpiar -ConfirmarBorradoDatos
```

El parámetro `-ConfirmarBorradoDatos` evita una eliminación involuntaria desde el script. No ejecutes esta última opción si todavía debes demostrar persistencia, reentrega o recuperación de la orden anterior. Si estás utilizando otro Compose de M06–M08, detenlo primero: sus puertos RabbitMQ entrarían en conflicto con el despliegue integral.

5.4 Taller dirigido: comprender, construir y ejecutar los seis contenedores

El script anterior proporciona una prueba integral repetible, pero el objetivo del alumno es poder explicar **por qué** funciona. Los siguientes pasos examinan los archivos que intervienen y reproducen las comprobaciones manualmente. Sigue el orden: los primeros pasos preparan la infraestructura y los últimos necesitan que los servicios estén levantados. No se crean soluciones C# nuevas ni se reemplazan los proyectos de referencia con ejemplos simplificados.

Paso 9.1 — Identificar cada proceso y su ubicación física

Antes de levantar el entorno, relaciona las direcciones del equipo Windows con las de la red privada de Docker. Esta representación sirve para interpretar por qué el navegador llega al gateway a través del puerto `5100` y, en cambio, Producción puede solicitar un recurso a `inventario-1:8080` sin salir de la red de contenedores:

```
PC del alumno: Windows + PowerShell + Visual Studio + Docker Desktop
  localhost:5100  -> Gateway (YARP)
  localhost:5105  -> Producción
  localhost:5106  -> Inventario 1
  localhost:5107  -> Inventario 2
  localhost:5672  -> RabbitMQ (AMQP)
  localhost:15672 -> RabbitMQ Management
           |
           v  Puertos publicados únicamente sobre 127.0.0.1
Red privada de Docker, llamada "aceria"
  gateway:8080     -> YARP
  produccion:8080  -> API y publicador outbox
  inventario-1:8080-> Consumidor y API
  inventario-2:8080-> Consumidor y API
  rabbitmq:5672    -> Broker
  sqlserver:1433   -> SQL Server Express
```

Abre `docker/compose.fullstack.yml` y localiza las seis entradas bajo `services`. Comprueba que `gateway` publica `127.0.0.1:5100:8080`, que las instancias de Inventario usan el mismo puerto interno `8080` y que `sqlserver` **no contiene una sección `ports`**. El valor anterior al último signo `:` en una publicación de puertos corresponde al anfitrión; el último corresponde al proceso dentro del contenedor.

Al terminar, deberías poder responder sin consultar el diagrama: ¿qué significa `localhost` para un navegador de Windows?, ¿qué significa para el proceso Producción dentro de Docker?, ¿y por qué SQL Server se identifica como `sqlserver` en las conexiones de EF Core? Si confundes estas tres ubicaciones, el diagnóstico posterior de conexiones será engañoso.

Paso 9.2 — Comprender qué se construye con el Dockerfile

Abre `docker/Dockerfile.Servicio` y sigue la secuencia desde arriba. La primera imagen contiene el SDK de .NET 8, que permite restaurar paquetes y publicar el proyecto. La instrucción `COPY` incorpora el código real de la etapa 8.4; `ARG PROYECTO` decide qué archivo `.csproj` se publicará. La segunda imagen contiene el runtime de ASP.NET Core, recibe los archivos publicados y no necesita incluir ni el SDK ni Visual Studio. Así se reduce lo que se distribuye para ejecutar cada servicio.

`ASPNETCORE_URLS=http://+:8080` indica a Kestrel dónde escuchar **dentro** del contenedor. Por sí solo, `EXPOSE 8080` documenta el puerto, pero no crea una entrada accesible desde Windows. Esa traducción se define en la sección `ports` de Compose.

Permaneciendo en `M09_DESPLIEGUE_DOCKER/PROYECTO`, construye únicamente la imagen de Producción:

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml build produccion
if ($LASTEXITCODE -ne 0) { throw 'Falló la compilación real de Producción.' }
docker image ls
```

Observa que se ejecutan `dotnet restore` y `dotnet publish` sobre `AceriaApp.Api`. La lista de imágenes debe incluir la construida para Producción; no debería depender de DLL preparadas a mano en el directorio del alumno. Si falla una referencia entre proyectos, revisa el **contexto relativo a la raíz del repositorio** antes de tocar el código. Reemplazar el servicio por una API vacía haría pasar una prueba diferente de la prevista.

Paso 9.3 — Diferenciar imagen, contenedor y servicio Compose

Acabas de generar una **imagen** de Producción. Eso no implica que exista un proceso escuchando: para que lo haya, Docker debe crear un **contenedor**. El **servicio Compose** es la declaración que determina qué imagen utilizar, qué variables recibe, a qué red se une y de qué otros servicios depende. Esto explica por qué `inventario-1` e `inventario-2` parten del mismo código pero son procesos independientes, con nombres y puertos externos diferentes.

Ejecuta estas dos consultas antes de `levantar` y repítelas después:

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml config --services
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml ps
```

La primera debe enumerar los seis servicios declarados aunque todavía no se hayan creado contenedores. La segunda muestra únicamente los contenedores del proyecto Compose. Tras el arranque, compara su estado con las verificaciones HTTP del Paso 9.7: «declarado», «iniciado» y «preparado para servir peticiones» son condiciones diferentes. Registra esta distinción en tu informe con un ejemplo tomado de la salida real.

Paso 9.4 — Seguir las variables de configuración hasta ASP.NET Core

El código de M08 puede ejecutarse en dos entornos porque su configuración no está grabada en los controladores ni en las clases del dominio. En Compose, `build.args` actúa durante la construcción de una imagen, mientras que `environment` establece valores para **cada proceso al arrancar**. El sistema de configuración de ASP.NET Core interpreta `__` como separación de niveles: `ConnectionStrings__AceriaProduccion` se corresponde con `ConnectionStrings:AceriaProduccion`.

Busca en `compose.fullstack.yml` las siguientes entradas y señala a qué servicio se asignan. No basta con encontrar sus nombres: explica qué dependencia real controla cada una.

1. `ConnectionStrings__AceriaProduccion` conecta el agregado DDD y el outbox con su base SQL.
2. `ConnectionStrings__AceriaInventario` dirige **ambas instancias** de Inventario hacia la misma base; de ese modo el balanceo no modifica los datos que observa el cliente.
3. `Comunicacion__RabbitMqUri` permite que el publicador y los consumidores utilicen el broker interno `rabbitmq:5672`.
4. `Comunicacion__InventarioUrl` establece el destino de las peticiones HTTP internas que hace Producción.
5. `ReverseProxy__Clusters__inventario-cluster__Destinations__inventario-1__Address` informa a YARP de uno de sus destinos de Inventario.

Para cada entrada, anota **nombre, servicio, valor de destino y síntoma si falla**. Por ejemplo, una URI de RabbitMQ que utilice `localhost:5672` dentro de Producción buscará el broker en el propio contenedor de Producción y no en `rabbitmq`. Una cadena SQL con `(localdb)\MSSQLLocalDB` fallará porque LocalDB solo está disponible en Windows. No se arregla instalando otra base de datos ni sustituyendo SQL Server por un proveedor simulado; se corrige el valor de la configuración.

La contraseña SQL viene de `.env` y se interpola en las conexiones al levantar Compose. No incluyas los valores secretos en la tabla que entregarás.

Paso 9.5 — Comprender redes, puertos y volúmenes

En el archivo Compose, `networks: aceria` crea la comunicación interna entre los servicios. Dentro de esa red Docker resuelve automáticamente `gateway`, `produccion`, `inventario-1`, `inventario-2`, `rabbitmq` y `sqlserver`. Por su parte, una declaración como `127.0.0.1:5106:8080` permite que una petición desde Windows llegue al puerto `8080` de Inventario 1 a través del `5106` local. Si el puerto `1433` de SQL Server no está publicado, el cliente no podrá conectarse a él directamente desde Windows por una dirección `localhost:1433`; las APIs sí pueden hacerlo desde la red interna.

Los volúmenes tienen una finalidad distinta. `sql-datos` aloja los ficheros persistentes de SQL Server y `rabbit-datos` los de RabbitMQ. Si el proceso de Producción se reinicia, sus datos de negocio no se pierden porque no residen en el sistema de archivos efímero de ese contenedor.

Comprueba la red, los volúmenes y los procesos antes y después del arranque:

```
docker network ls
docker volume ls
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml ps
```

Identifica el nombre que Compose ha asignado realmente a la red y a los volúmenes; puede llevar un prefijo de proyecto y no coincidir literalmente con la clave del YAML. Tras ejecutar `detener`, observa qué recursos permanecen; tras `bajar`, comprueba que los volúmenes siguen existiendo. No uses `docker volume prune` ni `down -v` en esta observación: destruirías precisamente aquello que pretendes verificar.

Paso 9.6 — Crear contraseñas de laboratorio y evitar exponer secretos

El archivo `.env.example` contiene una contraseña **publicable solo como ejemplo**, nunca adecuada para un despliegue real. En el Paso 5.1 ya la has copiado como `.env` y has elegido tu propia contraseña. **No vuelvas a copiar el ejemplo sobre ese archivo**, especialmente si SQL Server ya está levantado: modificar la variable del contenedor sin cambiar la contraseña almacenada en el servidor impediría conectar.

Si todavía no existe `.env`, créalo ahora; en caso contrario, abre el existente sin sobrescribirlo:

```
if (-not (Test-Path .\.env)) { Copy-Item .\.env.example .\.env }
notepad .\.env
git status --short
```

Comprueba que `.env` no figura como archivo pendiente de incorporar al repositorio. Para el laboratorio, las publicaciones de puertos están limitadas a `127.0.0.1`; esto reduce la exposición desde otras máquinas, pero no convierte las credenciales de prueba de RabbitMQ `curso / curso-local` en credenciales seguras. Del mismo modo, `TrustServerCertificate=True` acepta el certificado SQL de laboratorio sin validar una cadena de confianza como se haría en producción.

En tus capturas o informes no pegues el contenido de `.env` ni el resultado completo de `docker compose config`: la sustitución de variables puede mostrar la contraseña. Aporta, en su lugar, la salida de `config --quiet` y la constatación de que el archivo está ignorado por Git.

Paso 9.7 — Entender las comprobaciones de salud reales

Un proceso recién creado puede estar en estado `running` sin estar preparado. SQL Server necesita inicializar su motor y recuperar las bases; el `healthcheck` del Compose ejecuta una consulta `SELECT 1` con `sqlcmd`. RabbitMQ utiliza `rabbitmq-diagnostics -q ping`. Las tres instancias de API deben poder alcanzar sus dependencias para comunicar readiness; el gateway expone además una ruta de liveness que solo indica que **su propio proceso está vivo**.

Con los servicios arrancados, ejecuta:

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml ps
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml exec -T rabbitmq rabbitmq-diagnostics -q
ping
Invoke-RestMethod 'http://localhost:5100/health/live'
Invoke-RestMethod 'http://localhost:5105/health/ready'
Invoke-RestMethod 'http://localhost:5106/health/ready'
Invoke-RestMethod 'http://localhost:5107/health/ready'
```

Las consultas de las APIs deben devolver `Healthy` en la propiedad `estado`, mientras la respuesta del gateway confirma su liveness. Si `/health/live` responde pero el gateway no puede reenviar solicitudes, no hay contradicción: liveness y disponibilidad del sistema completo son señales distintas. Anota qué diferencia observas entre `docker compose ps`, la salud de SQL Server y las respuestas HTTP.

Ante un fallo, consulta primero el servicio afectado. Si SQL no está saludable, revisa sus logs y recursos disponibles; si falla la conexión AMQP, comprueba que RabbitMQ responde y que los nombres internos de Compose son correctos. No continúes con la creación de órdenes mientras los backends no estén preparados.

5.5 Taller dirigido: API Gateway, mensajería y resiliencia

Con los seis servicios en funcionamiento pasamos de la configuración a los resultados observables. Las siguientes operaciones emplean los **contratos HTTP de la misma AceriaApp** desarrollada en M02–M08: lectura de stock, creación de un agregado, publicación outbox, consumo inbox y verificación de persistencia. Ejecuta los pasos en una única sesión de PowerShell para mantener las variables del identificador de la orden; si cierras esa sesión, deberás recuperar los identificadores guardados, no inventar una orden sustituta.

Paso 9.8 — Comprobar que YARP realmente utiliza las dos instancias

Consulta Inventario **a través del gateway**, no accediendo directamente a los puertos `5106` y `5107`. Cada respuesta incorpora `X-Instancia`, cabecera que permite saber qué destino ha atendido la petición:

```
$gateway = 'http://localhost:5100'
$instancias = @{}
foreach ($i in 1..16) {
  $respuesta = Invoke-WebRequest "$gateway/api/v1/materiales/Acero"
  if ($respuesta.StatusCode -ne 200) { throw "Error HTTP $($respuesta.StatusCode)" }
  $nombre = [string]$respuesta.Headers['X-Instancia']
  if ($nombre) { $instancias[$nombre] = $true }
}
if (-not $instancias.ContainsKey('inventario-1') -or
    -not $instancias.ContainsKey('inventario-2')) {
  throw "No se observaron las dos instancias: $($instancias.Keys -join ', ')"
}
```

El código realiza varias solicitudes y registra los nombres de instancia observados. Para superar el paso deben aparecer `inventario-1` e `inventario-2`. De ese modo se demuestra que YARP utiliza ambos destinos en vez de limitarse a responder con una ruta fija. Las dos instancias comparten la base SQL, por lo que el valor de stock debe resultar coherente independientemente del destino que sirva el GET.

`RoundRobin` distribuye solicitudes sucesivas, pero no garantiza que dos procesos consuman idéntica CPU o que tengan la misma latencia. Registra la cabecera que corresponde a cada instancia y conserva una captura de las respuestas. Si falta uno de los destinos, comprueba los endpoints de readiness y la configuración `ReverseProxy__Clusters` antes de asumir que el balanceo está correcto.

Paso 9.9 — Crear una orden persistente mediante la API real

Ahora utilizaremos la API de Producción que ya se construyó en los módulos anteriores. Primero se crea una orden con 20 unidades de `Acero`; después se añade otra línea de 10 unidades de `Inoxidable`. El resultado debe ser un agregado en estado `Borrador` con dos líneas y cantidad total `30`, almacenado en SQL Server Express:

```
$cuerpo = @{ cliente='Axia Windows Docker'; material='Acero'; cantidad=20 } | ConvertTo-Json
$creada = Invoke-WebRequest -Uri "$gateway/api/v1/produccion/ordenes" -Method Post -ContentType 'application/
json' -Body $cuerpo -SkipHttpErrorCheck
if ($creada.StatusCode -ne 201) { throw "POST crear: $($creada.StatusCode)" }
$ordenId = [string](($creada.Content | ConvertFrom-Json).id)
$ordenUri = "$gateway/api/v1/produccion/ordenes/$ordenId"
$linea = @{ material='Inoxidable'; cantidad=10 } | ConvertTo-Json
$agregada = Invoke-WebRequest -Uri "$ordenUri/lineas" -Method Post -ContentType 'application/json' -Body
$linea -SkipHttpErrorCheck
if ($agregada.StatusCode -ne 200) { throw "POST línea: $($agregada.StatusCode)" }
$borrador = Invoke-RestMethod $ordenUri
if ($borrador.estado -ne 'Borrador' -or $borrador.cantidadTotal -ne 30) {
  throw 'El agregado SQL no conserva el total y el estado esperados.'
}
```

En este fragmento, `$gateway` procede del Paso 9.8. `ConvertTo-Json` crea la carga del POST, `201` indica creación y el campo `id` identifica la nueva orden. La URL `$ordenUri` utiliza ese identificador para añadir una línea y recuperar el objeto guardado. Antes de continuar, inspecciona la respuesta `$borrador` y comprueba sus datos, no solo el código HTTP.

Conserva el valor de `$ordenId`. En los próximos pasos lo utilizaremos para unir la transición de dominio, el mensaje asíncrono y la proyección de Inventario. Si en una comprobación posterior no aparece la orden original, **no generes otra orden para ocultar el fallo**: el ejercicio consiste precisamente en demostrar que un mismo agregado sobrevive al proceso completo.

Paso 9.10 — Comprobar que planificar emite un evento y prohíbe una segunda transición

La creación de la orden y su planificación son operaciones diferentes. El primer POST al endpoint de planificación debe aceptar la transición y dejar constancia del evento pendiente de publicación mediante el mecanismo **outbox**. Intentar planificar otra vez la misma orden debe producir un conflicto:

```
$plan = "$gateway/api/v1/comunicacion/ordenes/$ordenId/planificar"
$primera = Invoke-WebRequest -Uri $plan -Method Post -ContentType 'application/json' -Body '{}'
-SkipHttpErrorCheck
if ($primera.StatusCode -ne 202) { throw "Se esperaba 202: $($primera.Content)" }
$segunda = Invoke-WebRequest -Uri $plan -Method Post -ContentType 'application/json' -Body '{}'
-SkipHttpErrorCheck
if ($segunda.StatusCode -ne 409) { throw "Se esperaba 409: $($segunda.Content)" }
```

El código HTTP `202` significa que la operación ha sido aceptada; **no demuestra todavía** que Inventario haya recibido el evento. Por eso necesitamos el Paso 9.11. El `409`, por su parte, corresponde a la regla que impide aplicar dos veces esa transición de dominio. Esta protección no reemplaza la deduplicación del consumidor: una infraestructura de mensajería puede reentregar el mismo mensaje aunque solo se haya planificado la orden una vez.

Anota ambas respuestas y el identificador `$ordenId`. Si el segundo POST devuelve `202` o si el primero falla, detén la secuencia: continuar hasta la proyección haría difícil identificar dónde se rompió la regla del dominio.

Paso 9.11 — Acreditar la entrega asíncrona en el SQL de Inventario

La planificación aún debe viajar por RabbitMQ y ser procesada por Inventario. Como la entrega es asíncrona, la proyección puede tardar unos segundos en aparecer. El endpoint `/api/v1/planificaciones/{ordenId}` debe devolver `404` mientras no existe y `200` cuando ya se ha registrado en la base del consumidor:

```
$proyeccionUri = "$gateway/api/v1/planificaciones/$ordenId"
$proyeccion = $null
foreach ($intento in 1..75) {
  $respuesta = Invoke-WebRequest $proyeccionUri -SkipHttpErrorCheck
  if ($respuesta.StatusCode -eq 200) {
    $proyeccion = $respuesta.Content | ConvertFrom-Json
    break
  }
  if ($respuesta.StatusCode -ne 404) { throw "HTTP inesperado: $($respuesta.StatusCode)" }
  Start-Sleep -Seconds 2
}
if ($null -eq $proyeccion -or [string]$proyeccion.ordenId -ne $ordenId -or
    $proyeccion.cantidadTotal -ne 30) {
  throw 'No se acreditó el consumo outbox/inbox.'
}
$eventoId = [string]$proyeccion.eventoId
```

El bucle consulta periódicamente la proyección y abandona la espera cuando recibe `200`; cualquier otro código distinto de `404` es un problema que debe investigarse. La comprobación final exige que `ordenId` coincida con la orden creada y que la cantidad total siga siendo `30`. Almacena también `$eventoId` para la verificación posterior al reinicio.

Con este resultado podemos afirmar algo más fuerte que «hay conexiones a RabbitMQ»: se ha creado una orden, se ha publicado su evento, Inventario lo ha consumido y el efecto se ha persistido en SQL. Una cola vacía o un contenedor en estado `Up` no demuestran por sí solos ninguna de esas etapas.

Paso 9.12 — Interpretar las colas y los logs de RabbitMQ

Abre `http://localhost:15672/` con las credenciales de este laboratorio. Revisa las conexiones y canales, y localiza las colas asociadas al prefijo `aceria-docker-integral`. El publicador outbox actúa en Producción y los consumidores de Inventario reciben los mensajes; según cuándo observes la consola, la cola puede estar vacía porque un evento ya fue procesado correctamente.

Para reconstruir lo sucedido, relaciona la vista de RabbitMQ con los logs de los tres procesos:

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml logs --tail 100 rabbitmq
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml logs --tail 100 produccion
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml logs --tail 100 inventario-1
```

Comienza por Producción: ¿se planificó la orden y se preparó su publicación? Continúa por RabbitMQ: ¿está disponible el broker y se observan conexiones? Termina en Inventario: ¿se procesó el evento y se creó su proyección? La comprobación definitiva sigue siendo el GET del Paso 9.11, porque los logs y los contadores del broker pueden cambiar durante la ejecución.

Si no aparece la proyección, compara `WorkersActivos` y `Prefijo` en productor y consumidores, comprueba la URI AMQP interna y busca excepciones en los logs. No insertes manualmente una fila SQL ni elimines la cola como forma de hacer pasar el ejercicio.

Paso 9.13 — Simular la caída de Inventario 2 y la recuperación de YARP

Detendremos **solo** `inventario-2`. El gateway debe continuar atendiendo las consultas de materiales mediante `inventario-1`, que sigue teniendo acceso a la misma base. La detección de la caída puede requerir unos instantes; por eso no conviene convertir un retardo transitorio en un fallo definitivo tras una única petición.

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml stop inventario-2
$destinoSano = $false
foreach ($intento in 1..45) {
  try {
    $respuesta = Invoke-WebRequest "$gateway/api/v1/materiales/Acero" -TimeoutSec 5 -SkipHttpErrorCheck
    if ($respuesta.StatusCode -eq 200 -and
        [string]$respuesta.Headers['X-Instancia'] -eq 'inventario-1') {
      $destinoSano = $true
      break
    }
  } catch {
    # Puede haber una conexión transitoria mientras YARP detecta la caída.
  }
  Start-Sleep -Seconds 2
}
if (-not $destinoSano) { throw 'YARP no recuperó el servicio mediante Inventario 1.' }
foreach ($intento in 1..10) {
  $respuesta = Invoke-WebRequest "$gateway/api/v1/materiales/Acero" -TimeoutSec 10 -SkipHttpErrorCheck
  if ($respuesta.StatusCode -ne 200 -or
      [string]$respuesta.Headers['X-Instancia'] -ne 'inventario-1') {
    throw 'Una petición no fue atendida por el destino superviviente.'
  }
}
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml start inventario-2
```

La primera parte espera a que YARP deje de seleccionar el destino inactivo. Las diez consultas posteriores comprueban que la lectura permanece disponible mientras falta Inventario 2. Tras reiniciarlo, consulta `http://localhost:5107/health/ready` hasta que vuelva a estar preparado y repite el Paso 9.8 para verificar que **ambas instancias** participan de nuevo.

No reinicies el gateway para forzar el resultado: hacerlo ocultaría un defecto de detección y recuperación de destinos. Registra el comportamiento antes, durante y después de la interrupción.

Paso 9.14 — Detener RabbitMQ sin borrar mensajes ni bases

Ahora la dependencia que interrumpiremos es el broker. A diferencia del paso anterior, esto afecta a la comunicación asíncrona de Producción e Inventario; los procesos .NET pueden continuar vivos y sus endpoints de readiness pueden informar de degradación:

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml stop rabbitmq
try {
  $degradada = Invoke-WebRequest 'http://localhost:5106/health/ready' -TimeoutSec 5 -SkipHttpErrorCheck
  Write-Host "Readiness de Inventario sin broker: HTTP $($degradada.StatusCode)"
} catch {
  Write-Host "Inventario no responde con RabbitMQ detenido: $($_.Exception.Message)"
}
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml start rabbitmq
$brokerListo = $false
foreach ($intento in 1..45) {
  docker compose --env-file .\.env -f .\docker\compose.fullstack.yml exec -T rabbitmq rabbitmq-diagnostics -q
ping 2>$null | Out-Null
  if ($LASTEXITCODE -eq 0) { $brokerListo = $true; break }
  Start-Sleep -Seconds 2
}
if (-not $brokerListo) { throw 'RabbitMQ no recuperó su diagnóstico de salud.' }
```

Observa la respuesta HTTP de `/health/ready` y anota si comunica indisponibilidad mientras RabbitMQ está detenido. Tras iniciarlo, `rabbitmq-diagnostics -q ping` debe volver a responder correctamente; si la inicialización no ha concluido, repite la consulta después de unos segundos antes de diagnosticar un fallo permanente.

Detener el broker **no debe borrar el volumen `rabbit-datos`**. Para comprobar la recuperación completa crea y planifica una **nueva orden de prueba** y vuelve a exigir su proyección, aplicando los Pasos 9.9–9.11 a ese nuevo identificador. Guarda el identificador anterior por separado: se utilizará en el próximo paso para comprobar persistencia después de reiniciar las APIs. No confundir recuperación del puerto AMQP con entrega efectiva de mensajes.

Paso 9.15 — Reiniciar aplicaciones y demostrar persistencia real

Volvemos a la **orden original** del Paso 9.9, cuyo identificador guardamos en `$ordenId`. Reinicia las aplicaciones .NET sin borrar los volúmenes de SQL Server o RabbitMQ. Una vez recuperadas las comprobaciones de salud, solicita otra vez el agregado y la proyección de ese mismo evento:

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml restart inventario-1 inventario-2
produccion gateway
foreach ($puerto in @(5105,5106,5107)) {
  $preparado = $false
  foreach ($intento in 1..45) {
    try {
      $r = Invoke-WebRequest "http://localhost:$puerto/health/ready" -TimeoutSec 5 -SkipHttpErrorCheck
      if ($r.StatusCode -eq 200) { $preparado = $true; break }
    } catch {
      # La API aún está iniciándose: volver a consultar readiness.
    }
    Start-Sleep -Seconds 2
  }
  if (-not $preparado) { throw "La API del puerto $puerto no recuperó readiness." }
}
$ordenDespues = Invoke-RestMethod "$gateway/api/v1/produccion/ordenes/$ordenId"
$eventoDespues = Invoke-RestMethod "$gateway/api/v1/planificaciones/$ordenId"
if ($ordenDespues.estado -ne 'Planificada' -or $ordenDespues.cantidadTotal -ne 30) {
  throw 'La orden original no persistió.'
}
if ([string]$eventoDespues.eventoId -ne $eventoId) {
  throw 'La proyección original no conserva su ID.'
}
```

Comprueba tanto el estado `Planificada` como la cantidad `30` y el mismo `eventoId`. Si todo coincide, la evidencia no depende de los objetos que el proceso conservaba en memoria: proviene de datos recuperados de SQL Server. Una orden creada después del reinicio no serviría para esta prueba, ya que no demostraría que los datos anteriores sobrevivieron.

Paso 9.16 — Detener con conservación o destruir datos conscientemente

Al finalizar, decide qué resultado quieres preservar. Si el profesor va a revisar el laboratorio o todavía necesitas consultar los identificadores guardados, detén los procesos **sin destruir ningún volumen**:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion detener
```

Si además deseas retirar los contenedores y la red para liberar recursos, conservando las bases y las colas para otro arranque, utiliza en una operación separada:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion bajar
```

Existe una tercera operación, **destructiva**: únicamente cuando hayas guardado el informe, ya no necesites las órdenes ni los mensajes y quieras iniciar de cero el laboratorio, ejecuta **por separado**:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion limpiar -ConfirmarBorradoDatos
```

Las acciones no deben copiarse y ejecutarse indiscriminadamente como una secuencia. `detener` para los procesos, `bajar` elimina contenedores y red manteniendo volúmenes, y `limpiar` invoca `down -v`, que borra los volúmenes de este proyecto. Si el objetivo es demostrar persistencia, ejecuta solamente las primeras opciones y conserva los identificadores del Paso 9.15.

5.6 Matriz de evidencias de la práctica integral

La entrega se evalúa por observaciones reproducibles. Para cada fila, incluye la operación efectuada, el resultado recibido y el nombre del servicio que la produjo. Una captura de `docker ps` puede complementar el informe, pero nunca sustituir una petición HTTP o una lectura de datos.

Capacidad que debe demostrarse	**Evidencia concreta**
Windows con Docker Desktop y WSL 2	Equipo anfitrión Windows y `docker info` con `OSType=linux`
Construcción del código de M08	`Dockerfile.Servicio` y salida de `dotnet publish` al construir con Compose
SQL Server Express operativo	SQL `healthy` y recuperación de una orden tras el reinicio
RabbitMQ operativo	Diagnóstico AMQP correcto y evento recibido por Inventario
Gateway funcional	`/health/live`, reenvío HTTP y consultas de stock
Balanceo real	Cabecera `X-Instancia` con `inventario-1` e `inventario-2`
Orden DDD persistente	Creación `201`, adición de línea, estado `Borrador` y cantidad total 30
Outbox e inbox	Planificación `202` y proyección `200` del mismo `ordenId`
Regla de transición	Segunda planificación respondida con `409`
Degradación y recuperación	Consultas por Inventario 1 durante la caída de Inventario 2 y regreso de ambos destinos
Persistencia de la mensajería	Proyección conservada con el mismo `eventoId` tras reinicio
Registro reproducible	`.qa/docker-windows-e2e.json` con resultados de la ejecución sobre Windows

Entrega el JSON generado por `EJECUTAR_DOCKER_WINDOWS.ps1`, una tabla de los ensayos manuales, el `ordenId` y el `eventoId` usados para verificar persistencia, y una explicación propia de cómo se relacionan SQL Server, Producción, RabbitMQ, Inventario y YARP. **Nunca adjuntes el contenido real de `.env`.**

La revisión editorial de la práctica y la ejecución técnica son comprobaciones distintas. El script y las pruebas realizadas sobre Linux permiten verificar aspectos importantes del flujo, pero **no certifican por sí mismos** que Docker Desktop funcione correctamente en un equipo físico Windows. Hasta ejecutar allí `-Accion levantar` y guardar su reporte `PASS`, la certificación de ese entorno debe figurar como **PENDIENTE WINDOWS + DOCKER DESKTOP**.

6. Diagnóstico orientado a causas, no a atajos

Cuando el despliegue falle, sitúa primero el error en su dependencia. Un problema al iniciar el motor Docker no se resuelve modificando el controlador de órdenes; una conexión SQL incorrecta no se resuelve sustituyendo EF Core por un repositorio en memoria. La tabla siguiente relaciona los síntomas más probables con una comprobación que permite localizar su origen.

Síntoma	**Qué puede estar ocurriendo**	**Comprobación y siguiente paso**
`docker version` no conecta	Docker Desktop no está listo o se utiliza otro motor	Abre Docker Desktop y verifica `OSType=linux`
El puerto `5672` está ocupado	Sigue ejecutándose el Compose de M06–M08	Inspecciona `docker ps` y detén el broker del laboratorio anterior

Síntoma	Qué puede estar ocurriendo	Comprobación y siguiente paso
SQL Server no alcanza `healthy`	Credenciales, memoria, recuperación o herramienta `sqlcmd`	Lee los logs de `sqlserver` y revisa su `healthcheck` antes de arrancar las APIs
Una API entra en ciclo de reinicios	Fallo de conexión, migración o configuración .NET	Consulta los logs de `produccion` o `inventario-1` y comprueba SQL
En Docker aparece `(localdb)`	Se heredó una cadena propia de Windows	Revisa `ConnectionStrings__*`: debe utilizar `sqlserver,1433`
RabbitMQ no es accesible desde Producción	Se usa `localhost` como si fuese el nombre del broker	Revisa `Comunicacion__RabbitMqUri` y ejecuta el diagnóstico del contenedor `rabbitmq`
El gateway devuelve `503`	Destinos no preparados o configuración YARP incorrecta	Consulta directamente los `/health/ready` de los backends y revisa las direcciones `ReverseProxy`
No aparece la proyección de Inventario	Publicador/consumidor detenidos, prefijos distintos o mensaje pendiente	Comprueba `WorkersActivos`, `Prefijo`, logs y GET de la proyección del mismo `ordenId`
Tras reiniciar desaparece una orden	Se recreó la base, se borró el volumen o se cambió el identificador	Revisa la acción de limpieza, el volumen `sql-datos` y el `ordenId` original
Un test pasa sin depender de SQL real	La prueba utiliza un sustituto que oculta errores de integración	Examina el proveedor y la conexión efectiva y revisa el resumen de xUnit

El cierre exige coherencia entre **manual, código y observación**. No certificamos la comunicación asíncrona únicamente por ver un broker activo; tampoco damos por válida la persistencia por arrancar SQL Server. Son las mismas respuestas e identificadores antes y después de las interrupciones los que permiten concluir que AceriaApp funciona como sistema distribuido.

Límite de validación: los tres entornos descritos en este módulo deben distinguirse en el informe: Windows + LocalDB, Windows + RabbitMQ en Docker con APIs en el anfitrión y Windows + despliegue integral en Docker Desktop. La certificación final del último escenario requiere una ejecución real sobre Windows; la corrección del texto y los ensayos de CI no reemplazan esa evidencia.